Print copy of official Microsoft Learn documentation

Source: https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/overview
Exported: 2026-10-14. Source metadata date: 2026-02-06T00:00:00Z.

Microsoft and documentation contributors. Article wording and code for the requested version are retained; web navigation is removed and print formatting is added. This locally generated PDF is not a Microsoft-published PDF book or a complete product manual. Source-page attribution and notices apply.

Documentation license and terms: Microsoft Learn terms; .NET documentation: CC BY 4.0 license. Check the source for specific attribution or exceptions.

A tour of the C# language

This article gives you a high-level overview of the C# language. Whether you're writing your first program or you're an experienced developer exploring C#, you'll find the key concepts and features here.

> Tip
>
> **New to programming?** Skim this article for the big picture, then start the beginner tutorials to write code hands-on.
>
> **Coming from Java, JavaScript, or Python?** Read the tips for Java, JavaScript, or Python developers alongside this article.
>
> **Experienced developer new to C#?** This article covers what makes C# distinctive. See What you can build with C# to find the workload that fits your goals.

The C# language is the most popular language for the .NET platform, a free, cross-platform, open source development environment. C# programs can run on many different devices, from Internet of Things (IoT) devices to the cloud and everywhere in between. You can write apps for phone, desktop, and laptop computers and servers. See What you can build with C# for an overview of application types.

C# is a cross-platform general purpose language that makes developers productive while writing highly performant code. C# has broad support in the ecosystem and many workloads. Based on object-oriented principles, it incorporates many features from other paradigms, not least functional programming. Low-level features support high-efficiency scenarios without writing unsafe code. Most of the runtime and libraries are written in C#, and advances in C# often benefit all .NET developers.

C# is in the C family of languages. C# syntax is familiar if you used C, C++, JavaScript, TypeScript, or Java. Like C and C++, semicolons (`;`) define the end of statements. C# identifiers are case-sensitive. C# has the same use of braces, `{` and `}`, control statements like `if`, `else`, and `switch`, and looping constructs like `for` and `while`. C# also has a `foreach` statement for any collection type.

Hello world

The "Hello, World" program traditionally introduces a programming language. Here's the program in C#:

```
// This line prints "Hello, World"
Console.WriteLine("Hello, World");
```

The line starting with `//` is a *single line comment*. C# single line comments start with `//` and continue to the end of the current line. C# also supports *multi-line comments*. Multi-line comments start with `/*` and end with `*/`. The `WriteLine` method of the `Console` class, which is in the `System` namespace, produces the output of the program. The standard class libraries provide this class, and every C# program automatically references these libraries by default. Another program form requires you to declare the containing class and method for the program's entry point. The compiler synthesizes these elements when you use top-level statements.

This alternative format is still valid and contains many of the basic concepts in all C# programs. Many existing C# samples use the following equivalent format:

```
using System;
namespace TourOfCsharp;

class Program
{
    static void Main()
    {
        // This line prints "Hello, World"
        Console.WriteLine("Hello, World");
    }
}
```

The preceding "Hello, World" program starts with a `using` directive that references the `System` namespace. Namespaces provide a hierarchical means of organizing C# programs and libraries. Namespaces contain types and other namespaces. For example, the `System` namespace contains many types, such as the `Console` class referenced in the program, and many other namespaces, such as `IO` and `Collections`. A `using` directive that references a given namespace enables unqualified use of the types that are members of that namespace. Because of the `using` directive, the program can use `Console.WriteLine` as shorthand for `System.Console.WriteLine`. In the earlier example, that namespace was implicitly included.

> Note
>
> The preceding program declares the `Program` class with a single member: the method named `Main`. Many existing C# samples and tutorials use this format. By convention, when there are no top-level statements, a static method named `Main` serves as the entry point of a C# program. Both formats compile to the same code. You don't need to use this format—file-based apps and top-level statements are simpler ways to get started.

File-based apps

C# is a *compiled* language. In most C# programs, you use the `dotnet build` command to compile a group of source files into a binary package. Then, you use the `dotnet run` command to run the program. (You can simplify this process because `dotnet run` compiles the program before running it if necessary.) These tools support a rich language of configuration options and command-line switches. The `dotnet` command line interface (CLI), which is included in the .NET SDK, provides many tools to generate and modify C# files.

Beginning with C# 14 and .NET 10, you can create *file-based apps*, which simplifies building and running C# programs. You use the `dotnet run` command to run a program contained in a single `*.cs` file. For example, if the following code is stored in a file named `hello-world.cs`, you can run it by typing `dotnet run hello-world.cs`:

```
#!/usr/bin/env dotnet

Console.WriteLine("Hello, World!");
```

The first line of the program contains the `#!` sequence (shebang) for unix operating systems. This enables executing the file directly using its name when the *execute* (`+x`) permission is set on the file. For example, you can run the C# file directly from the command line:

```
./hello-world.cs
```

The source for these programs must be a single file, but otherwise all C# syntax is valid. You can use file-based apps for small command-line utilities, prototypes, or other experiments.

Familiar C# features

C# is approachable for beginners yet offers advanced features for experienced developers writing specialized applications. You can be productive quickly. You can learn more specialized techniques as you need them for your applications.

C# apps benefit from the runtime's automatic memory management. C# apps also use the extensive runtime libraries provided by the .NET SDK. Some components are platform independent, like file system libraries, data collections, and math libraries. Others are specific to a single workload, like the ASP.NET Core web libraries or the .NET MAUI UI library. A rich open source ecosystem on NuGet augments the libraries that are part of the runtime. These libraries provide even more components you can use.

C# is a *strongly typed* language. Every variable you declare has a type known at compile time. The compiler, or editing tools, tell you if you're using that type incorrectly. You can fix those errors before you ever run your program. Fundamental data types are built into the language and runtime: value types like `int`, `double`, `char`, reference types like `string`, arrays, and other collections. As you write your programs, you create your own types. Those types can be `struct` types for values, or `class` types that define object-oriented behavior. You can add the `record` modifier to either `struct` or `class` types so the compiler synthesizes code for equality comparisons. You can also create `interface` definitions, which define a contract, or a set of members, that a type implementing that interface must provide. You can also define generic types and methods. Generics use *type parameters* to provide a placeholder for an actual type when used.

As you write code, you define functions, also called methods, as members of `struct` and `class` types. These methods define the behavior of your types. You can overload methods with different numbers or types of parameters. Methods can optionally return a value. In addition to methods, C# types can have properties, which are data elements backed by functions called *accessors*. C# types can define events, which allow a type to notify subscribers of important actions. C# supports object oriented techniques such as inheritance and polymorphism for `class` types.

C# apps use exceptions to report and handle errors. This practice is familiar if you used C++ or Java. Your code throws an exception when it can't do what was intended. Other code, no matter how many levels up the call stack, can optionally recover by using a `try` - `catch` block.

> Tip
>
> To learn more about types, methods, and exceptions, visit the C# fundamentals section. It covers the type system, object-oriented programming, and exception handling in depth.

Distinctive C# features

Some elements of C# might be less familiar.

C# provides pattern matching. Those expressions enable you to inspect data and make decisions based on its characteristics. Pattern matching provides a great syntax for control flow based on data. The following code shows how methods for the boolean *and*, *or*, and *xor* operations could be expressed using pattern matching syntax:

```
public static bool Or(bool left, bool right) =>
    (left, right) switch
    {
        (true, true) => true,
        (true, false) => true,
        (false, true) => true,
        (false, false) => false,
    };

public static bool And(bool left, bool right) =>
    (left, right) switch
    {
        (true, true) => true,
        (true, false) => false,
        (false, true) => false,
        (false, false) => false,
    };
public static bool Xor(bool left, bool right) =>
    (left, right) switch
    {
        (true, true) => false,
        (true, false) => true,
        (false, true) => true,
        (false, false) => false,
    };
```

Pattern matching expressions can be simplified by using _ as a catchall for any value. The following example shows how you can simplify the *and* method:

```
public static bool ReducedAnd(bool left, bool right) =>
    (left, right) switch
    {
        (true, true) => true,
        (_, _) => false,
    };
```

The preceding examples also declare *tuples*, lightweight data structures. A *tuple* is an ordered, fixed-length sequence of values with optional names and individual types. You enclose the sequence in `(` and `)` characters. The declaration `(left, right)` defines a tuple with two boolean values: `left` and `right`. Each switch arm declares a tuple value such as `(true, true)`. Tuples provide convenient syntax to declare a single value with multiple values of any type.

Collection expressions provide a common syntax to provide collection values. You write values or expressions between `[` and `]` characters and the compiler converts that expression to the required collection type:

```
int[] numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
List<string> names = ["Alice", "Bob", "Charlie", "David"];

IEnumerable<int> moreNumbers = [.. numbers, 11, 12, 13];
IEnumerable<string> empty = [];
```

The previous example shows different collection types that you can initialize by using collection expressions. One example uses the `[]` empty collection expression to declare an empty collection. Another example uses the `..` *spread element* to expand a collection and add all its values to the collection expression.

You can use *index* and *range* expressions to retrieve one or more elements from an indexable collection:

```
string second = names[1]; // 0-based index
string last = names[^1]; // ^1 is the last element
int[] smallNumbers = numbers[0..5]; // elements at indexes 0 to 4
```

The `^` index indicates *from the end* rather than from the start. The `^0` element is one past the end of the collection, so `^1` is the last element. The `..` in a range expression denotes the range of elements to include. The range starts with the first index and includes all elements up to, but not including, the element at the last index.

For more information about index and range expressions, see the Explore indexes and ranges article.

Language integrated query (LINQ) provides a common pattern-based syntax to query or transform any collection of data. LINQ unifies the syntax for querying in-memory collections, structured data like XML or JSON, database storage, and even cloud based data APIs. You learn one set of syntax and you can search and manipulate data regardless of its storage. The following query finds all students whose grade point average is greater than 3.5:

```
var honorRoll = from student in Students
                where student.GPA > 3.5
                select student;
```

The preceding query works for many storage types represented by `Students`. It could be a collection of objects, a database table, a cloud storage blob, or an XML structure. The same query syntax works for all storage types.

The Task based asynchronous programming model enables you to write code that reads as though it runs synchronously, even though it runs asynchronously. It utilizes the `async` and `await` keywords to describe methods that are asynchronous, and when an expression evaluates asynchronously. The following sample awaits an asynchronous web request. When the asynchronous operation completes, the method returns the length of the response:

```
public static async Task<int> GetPageLengthAsync(string endpoint)
{
    var client = new HttpClient();
    var uri = new Uri(endpoint);
    byte[] content = await client.GetByteArrayAsync(uri);
    return content.Length;
}
```

C# also supports an `await foreach` statement to iterate a collection backed by an asynchronous operation, like a GraphQL paging API. The following sample reads data in chunks, returning an iterator that provides access to each element when it's available:

```
public static async IAsyncEnumerable<int> ReadSequence()
{
    int index = 0;
    while (index < 100)
    {
        int[] nextChunk = await GetNextChunk(index);
        if (nextChunk.Length == 0)
        {
            yield break;
        }
        foreach (var item in nextChunk)
        {
            yield return item;
        }
        index++;
    }
}
```

Callers can iterate the collection by using an `await foreach` statement:

```
await foreach (var number in ReadSequence())
{
    Console.WriteLine(number);
}
```

Finally, as part of the .NET ecosystem, you can use Visual Studio or Visual Studio Code with the C# Dev Kit. These tools provide a rich understanding of C#, including the code you write. They also provide debugging capabilities.

> Tip

To learn more about pattern matching, LINQ, and async programming, see the pattern matching, LINQ overview, and asynchronous programming sections.

Next steps

This article provided a quick tour of the C# language. Here's where to go next, depending on your experience:

- **Start coding**: Work through the beginner tutorials to learn C# step by step.
- **Go deeper**: Visit the C# fundamentals section for detailed coverage of the type system, object-oriented programming, and error handling.
- **Build something**: Explore what you can build with C# to find ideas that interest you.
- **Coming from another language?** Read the guides for Java, JavaScript, or Python developers.
- **Build file-based apps**: Learn how to build file-based apps for small programs and prototypes.